Does Graph Geometry Help Forecasting the S&P 100?

Antonio Lorenzo Díaz Meco and Javier Ríos Montes
Final Project — Geometric Deep Learning
GitHub Repository

Abstract

Write the abstract here.

1 Introduction

Stock forecasting is a challenging task that, due to its economic implications, has been an attractive area of research for decades. In specific, AI and machine learning techniques have been widely applied to this problem, with varying degrees of success. Despite the advances in the matter, many current approaches are limited to using technical analysis to capture historical trends of each stock price. In this paper, we explore the potential benefits of incorporating graph geometry into stock forecasting models, specifically focusing on the S&P 100 index. We decide to focus on this subset of the stock market to reduce the complexity of the problem and to be able to focus on the relationships between the stocks, which is where we expect graph geometry to provide the most value.

In indeces such as the S&P 100, there are important relationships between individual stocks, such as shared market sectors or financial indicators, that may not be fully captured by traditional forecasting models and that could be better modeled using graph-based approaches. By representing the stocks as nodes in a graph and their relationships as edges, we can leverage geometric deep learning techniques to capture these interactions and, potentially, improve forecasting performance

In order to evaluate the effectiveness of graph geometry in stock forecasting, we first implement two baseline models: a Random Forest and an LSTM. These models will serve as benchmarks to compare against a Graph Neural Network (GNN). The choice of these architectures is motivated by their widespread use in time series forecasting and their ability to capture different aspects of the data. The Random Forest is a powerful ensemble method that can capture non-linear relationships (and, given a sufficiently informative feature set, obtain performance comparable to more complex models), while the LSTM is a type of recurrent neural network that is particularly well-suited for sequential data (in this case, the temporal evolution of the index). The GNN, on the other hand, is designed to capture the relationships between stocks by modeling them as a graph, which is where we expect to see the most significant improvements in forecasting performance.

We model the relationships between stocks using a graph structure, where nodes represent individual stocks and edges represent relationships, mainly shared market sectors, correlations between stock returns, and divergences in the returns' distributions. The problem is thus framed as a node classification task, in which we aim to predict the direction of the stock price movement (up, neutral, or down) for each stock in the index within a given time frame. We evaluate the performance of the models using standard metrics such as accuracy, precision, recall, and F1-score, and we analyze the results to understand the impact of incorporating graph geometry into the forecasting process.

2 Related Work

Financial forecasting surged as a research area in the late 19th century, with the introduction of statistical methods (such as those developed by Irving Fisher) to analyze market trends. It wasn't, though, until the mid-20th century that advances in computing power allowed for the application

of more complex models. All in all, we can roughly divide the evolution of financial forecasting into three main eras: the pre-computing era, the statistical era, and the machine learning era. The pre-computing era was characterized by the use of simple heuristics and technical analysis to predict stock prices.

The statistical era saw the introduction of parametric models that, while much more expressive than the previous heuristics, were based upon explicit assumptions about the underlying data distribution [2]. For example, ARIMA models the conditional mean of the time series as a linear function of past observations and errors, while GARCH models the conditional variance as a function of past squared errors and variances.

The machine learning era, currently under process, is characterized by the use of non-parametric models that can capture complex relationships in the data without making strong assumptions about the underlying distribution. The last decades have seen a surge in implementations of machine learning models for financial forecasting, including general-purpose models such as Random Forests [3] and time-series specific models such as LSTMs [8] and Gateted Recurrent Units (GRUs) [1]. With the introduction of the Transformer architecture [9], attention-based architectures surged as a promising alternative to recurrent models, as they allow for long-range dependencies to be captured more effectively and in a more parallelizable way. In the specific context of stock forecasting, attention-based models have been shown to outperform recurrent models in certain settings [4].

Last, there has been a growing interest in the use of graph-based models for financial forecasting. Lead by the assumption that the relationships between stocks can provide valuable information for forecasting (and may not be fully captured by traditional models, as they lack the necessary inductive bias to model these relationships), the last years have witnessed a boom in the implementation of GNN-based models for stock forecasting. These range from purely graph-based models [6] to hybrid models that combine graph-based and sequence-based architectures [5, 7]. Added to this, the hybrid paradigm has evolved from simple combinations of sequence-based and graph-based models to more complex architectures in which the graph structure is integrated into the sequence-based model itself, such as TCGNs (Temporal Convolutional Graph Networks) [10].

In general, the results in the literature suggest that incorporating graph geometry into stock forecasting models can lead to improved performance, especially when the relationships between stocks are strong and informative. This does come with a caveat, though, as the performance of graph-based models can be sensitive to the choice of graph structure and the quality of the data used to construct the graph (more so than with simpler models as LSTMs or Random Forests). In this sense, we hypothesize that, due to the variety in graphs we may construct on a given index (for example, based on different financial indicators or with different thresholds for edge creation), the performance of the GNN may vary significantly depending on the graph structure used. Thus, the results of this project will additionally contribute to the understanding of the impact of graph structure on forecasting performance, which is an important aspect to consider when applying graph-based models to financial forecasting.

3 Background

4 Method

5 Experimental Setup

The data used in this project consists of the historical stock prices of the companies included in the S&P 100 index, along with relevant financial indicators, depending on the specific model. We use the following variables, common for all models, as features for each stock:

1. Identifiers and Basic Information:
    - Ticker symbol
    - Date
    - Adjusted closing price
    - Daily Log Return: $\log\left(\frac{P_t}{P_{t-1}}\right)$, where P_t is the adjusted closing price at time t.

- Future Weekly Log Return: $\sum_{i=1}^{5} \log\left(\frac{P_{t+i}}{P_{t+i-1}}\right)$, used only to define the prediction target 5 trading days ahead.
- Class label: a categorical variable indicating the direction of the stock price movement (up, neutral, down), defined in terms of the sign of the future Weekly Log Return.

2. Volume:
   - Daily trading volume
   - Normalized trading volume: the daily trading volume divided by the 20-day rolling average trading volume of the same stock.
3. Technical Indicators:
   - Weekly Moving Average (5MA): the average of the adjusted closing price over the past 5 trading days.
   - Monthly Moving Average (20MA): the average of the adjusted closing price over the past 20 trading days.
   - Relative Strength Index (RSI): a momentum oscillator that measures the speed and change of price movements, calculated as $RSI = 100 - \frac{100}{1+RS}$, where RS is the average gain of up periods divided by the average loss of down periods over a specified time frame (14 trading days in our case).
   - Rolling Volatility: the standard deviation of the daily log returns over a rolling window of 20 trading days.
4. Structural Information:
   - Market sector: a categorical variable indicating the market sector to which the stock belongs (e.g., Technology, Healthcare, Financials, etc.).
   - Market Cap: a continuous variable indicating the market capitalization of the company (in dollars).

In addition to this, for the LSTM model, we include the sequence of the past 20 days of the above features as input, while for the GNN model, we construct two different graph structures based on the following relationships between stocks:

- A correlation graph, where each stock is connected to its top-k most correlated neighbors using training-set stock returns, with additional sector-similarity edges.
- A distributional-similarity graph, where each stock is connected to its top-k closest neighbors according to the Jensen-Shannon divergence between return distributions, with additional sector-similarity edges.

All data was obtained from Yahoo Finance using the `yfinance` Python library, for the period between January 4, 2021 and December 31, 2025. The choice of this period is motivated by two main reasons: (i) 5 years is a common time frame used in the literature for stock forecasting, as it provides a good balance between the amount of data available for training and the relevance of the data (as older data may not be as relevant for forecasting future stock movements), and (ii) it avoids, at least theoretically, the impact of the COVID-19 pandemic on the stock market, which introduced a significant amount of noise and structural changes in the market that could have affected the performance of the models [1].

5.1 Data and Preprocessing

In order to obtain the information that we are to feed into our models, we followed the following steps:

- We obtained the list of stocks in the S&P 100 index, along with their corresponding ticker symbols. In specific, we filtered this list to the stocks that have been in the index for the entire period of time considered (January 4, 2021 to December 31, 2025), which resulted in a total of 88 stocks. Even though this filtering step reduces the number of stocks we

[1] After careful analysis, we assumed that the turmoil caused by the COVID-19 pandemic in the stock market had a significant impact until the end of 2020, and that, from January 2021 onwards, the market had stabilized enough for the data to be considered as not significantly affected by the pandemic.

are considering, we consider the reduction small enough to not significantly affect the performance of the models, while it allows us to avoid dealing with structural changes in the index.

- For each stock, we obtained the data mentioned in the previous section, computing the necessary features. As to the labels, we decided to define a threshold ϵ on the future Weekly Log Return to determine the class labels, in order to avoid labeling small fluctuations as significant movements. Specifically, we defined the class labels as follows:
  - Up: if the future Weekly Log Return is greater than ϵ
  - Neutral: if the future Weekly Log Return is between $-\epsilon$ and ϵ
  - Down: if the future Weekly Log Return is less than $-\epsilon$

  We set ϵ to 0.01, which corresponds to a 1% change in the stock price over a week. Though this threshold is rather arbitrary, it yields a good balance between the number of samples in each class, as mentioned in A.2.
- We split the data into training, validation, and test sets, using a chronological split to avoid data leakage. Specifically, training data ends on December 31, 2023, validation data ends on December 31, 2024, and the remaining observations are used for testing. This divides the dataset into roughly 3, 1, and 1 years, respectively, and makes the splits comparable across models.

  This is done in different ways for the different models, though. For the Random Forest, we assign the split in the tabular dataset according to the date of each sample. For the LSTM, we create sequences of 20 days for each stock and assign their split according to the reference date of each sequence. For the GNN, we create the graph structure using the data from the training set, and then split the snapshots of the graph (this is, every node and edge at a given time step).

These preprocessing steps are implemented as a reproducible pipeline that can be executed from the repository root with `uv run python -m src.data.run_pipeline`.

5.2 Models and Hyperparameters

The GNN experiments are trained using AdamW with weight decay, cross-entropy loss, gradient clipping, a fixed random seed, and early stopping based on validation macro-F1. The best checkpoint according to validation macro-F1 is saved during training and reloaded before computing the final test score.

6 Results and Discussion

7 Conclusions and Future Work

Contribution Statement

While both of us developed the code together and contributed to the writing of the report, there are certain sections that were implemented primarily by each one of us.

- **Antonio Lorenzo Díaz Meco**: Code for the 3 models (Random Forest, LSTM, and GNN), and experimental setup. Writing of the Method and Experimental Setup.
- **Javier Ríos Montes**: Code for data preprocessing and Repository structure. Writing of the Introduction, Related Work, Background, Results and Discussion, and Conclusions and Future Work.

AI Tools Disclosure Statement

In order to complete the project, we used the following AI tools:

- **GitHub Copilot**: Mainly during the data preprocessing stage, to write code for feature engineering and data cleaning. Additionally, it was marginally used throughout the whole codebase to solve minor coding issues and bugs.
- **Perplexity AI**: In our personal experience, Comet is the best one for brain-storming and generating ideas, as well as for improve the legibility of the report. We used it mainly for these purposes, and to a lesser extent for writing assistance, such as recovering useful sources and references for our work.
- **ChatGPT Codex**: Given its ability to analyze the complete structure of a codebase, we used it to review the code and suggest points of failure in the connection between individual components, as well as to suggest improvements in the code structure and modularity.
- **Claude Code**: While Comet is better for brainstorming and writing, we found that Claude is better for code generation. We used it mainly to, given the pseudocode of an architecture or a function, generate the corresponding code in Python.

References

[1] Suresh Chavhan, Praveen Raj, Prayas Raj, Ashit Kumar Dutta, and Joel J.P. C. Rodrigues. Deep learning approaches for stock price prediction: A comparative study of lstm, rnn, and gru models. In *2024 9th International Conference on Smart and Sustainable Technologies (SpliTech)*, pages 01–06, 2024. doi: 10.23919/SpliTech61897.2024.10612666. URL https://ieeexplore.ieee.org/document/10612666.

[2] Noelia Crespo González. Predicción de series temporales financieras con modelos de machine learning. Unpublished, 2025. URL https://oa.upm.es/91012/.

[3] Richard A. Davis and Mikkel S. Nielsen. Modeling of time series using random forests: theoretical developments, 2020. URL https://arxiv.org/abs/2008.02479.

[4] Yawei Li, Shuqi Lv, Xinghua Liu, and Qiuyue Zhang. Incorporating transformers and attention networks for stock movement prediction. *Complexity*, 2022:1–10, 02 2022. doi: 10.1155/2022/7739087. URL https://www.researchgate.net/publication/358897376_Incorporating_Transformers_and_Attention_Networks_for_Stock_Movement_Prediction.

[5] Antonio Longa, Veronica Lachi, Gabriele Santin, Monica Bianchini, Bruno Lepri, Pietro Lio, Franco Scarselli, and Andrea Passerini. Graph neural networks for temporal graphs: State of the art, open challenges, and opportunities, 2023. URL https://arxiv.org/abs/2302.01018.

[6] Daiki Matsunaga, Toyotaro Suzumura, and Toshihiro Takahashi. Exploring graph neural networks for stock market predictions with rolling window analysis, 2019. URL https://arxiv.org/abs/1909.10660.

[7] Meet Satishbhai Sonani, Atta Badii, and Armin Moin. Stock price prediction using a hybrid lstm-gnn model: Integrating time-series and graph-based analysis, 2025. URL https://arxiv.org/abs/2502.15813.

[8] Koushik Sutradhar, Sourav Sutradhar, Iqbal Ahmed Jhimel, Suneet Kumar Gupta, and Mohammad Monirujjaman Khan. Stock market prediction using recurrent neural network's lstm architecture. In *2021 IEEE 12th Annual Ubiquitous Computing*, pages 0541–0547, 2021.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL `https://arxiv.org/abs/1706.03762`.

[10] Tianye Wang. Enhancing stock market prediction with temporal graph neural networks and large language model-based explainability. *Procedia Computer Science*, 274:147–160, 2025. ISSN 1877-0509. doi: https://doi.org/10.1016/j.procs.2025.12.015. URL `https://www.sciencedirect.com/science/article/pii/S1877050925037366`. 22nd International Multidisciplinary Modeling and Simulation Multiconference (I3M 2025).

A Important Details

A.1 Sector Information

The 88 stocks that we are considering in our dataset belong to the following market sectors:

- Financial Services: 16 stocks
- Healthcare: 14 stocks
- Technology: 12 stocks
- Industrials: 12 stocks
- Consumer Defensive: 10 stocks
- Consumer Cyclical: 9 stocks
- Communication Services: 7 stocks
- Energy: 3 stocks
- Utilities: 3 stocks
- Real Estate: 2 stocks

A.2 Class Distribution

The distribution of the class labels (up, neutral, down), given $\epsilon = 0.01$, is as follows:

- Up: 41.3%
- Neutral: 24.85%
- Down: 33.85%

This distribution is relatively balanced and, moreover, may reflect a certain intuition about the stock market, in which, over a week, stocks are more likely to go up or down than to remain neutral (as small fluctuations are not labeled as significant movements) and, overall, there is a slightly higher probability of stocks going up than down (which may be due to the general growth trend of the stock market over time). This distribution should allow the models to learn effectively without being biased towards a particular class.